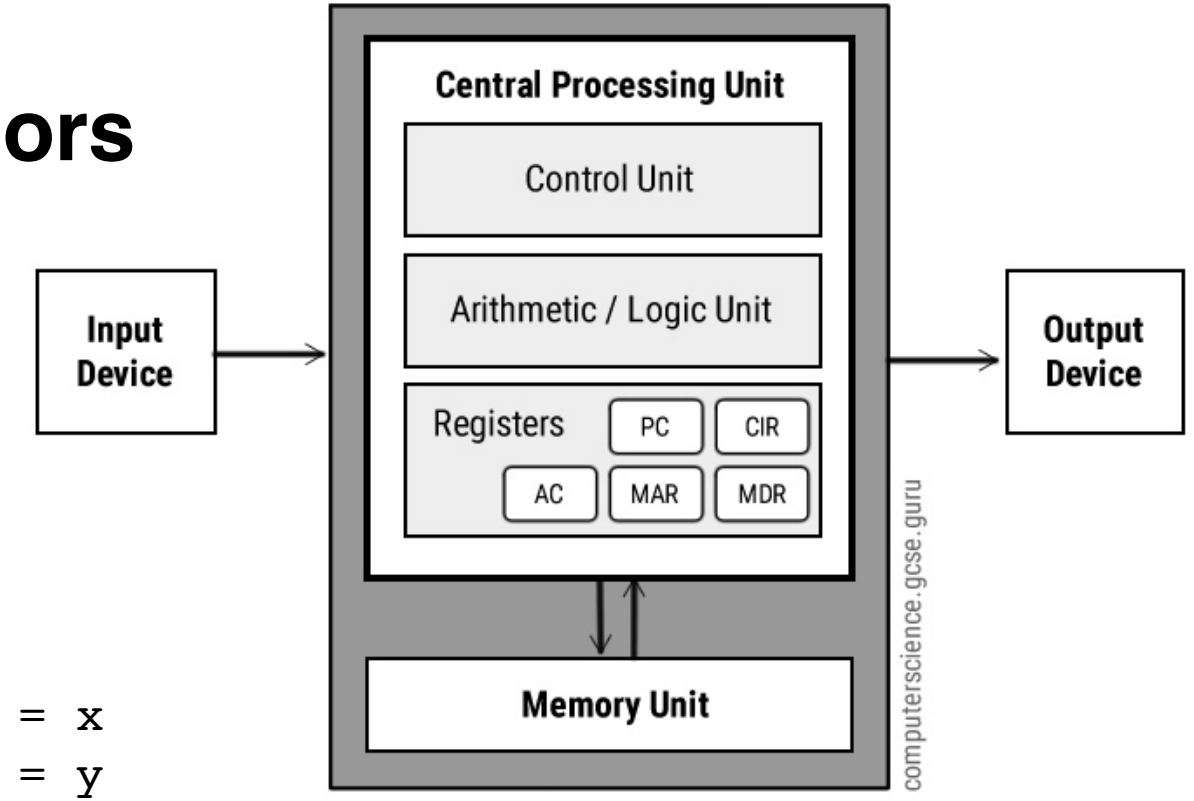
Processor Vulnerabilities

Heorhii Ambartsumov

What we expect from processors

- We send some instructions and we execute them.
- Everything should work in order and be super intuitive.




```
movl x, %eax                    # %eax = x
movl y, %ebx                    # %ebx = y

addl %ebx, %eax                 # %eax = %eax + %ebx (aka x + y)
subl $3, %eax                   # %eax = %eax - 3
imull $2, %eax                  # %eax = %eax * 2

cmpl $20, %eax                  # set flags as if (%eax - 20)
jg greater_than_20              # jump if %eax > 20
```

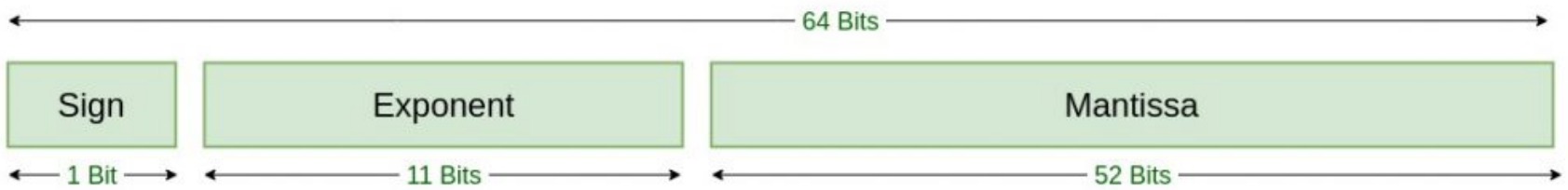
Still within expectations: Kocher timing attacks

- RSA encryption relies on computation of expressions like $m^k \bmod n$
- A fast way to compute that is

```
def fast_exp(base, exponent, modulus):
    result = 1
    base = base % modulus
    while exponent > 0:
        if exponent & 1:  # extra work when bit = 1
            result = (result * base) % modulus
        base = (base * base) % modulus
        exponent >>= 1
    return result
```

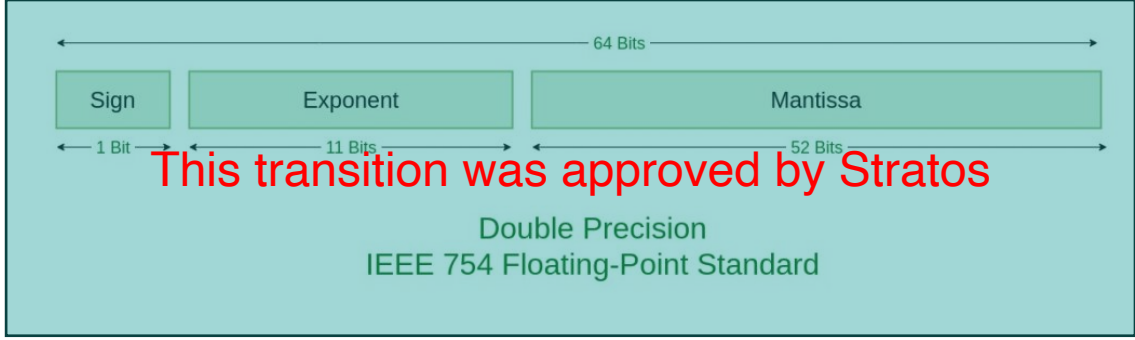
If you want a demo: https://github.com/zksecurity/timing-notebook

Failure of expectations: floats



Double Precision
IEEE 754 Floating-Point Standard

Failure of expectations: floats



- 2^53+1 = 2^53 – adding a small number to a large number might not actually change the result.
- On x86-64,

```
# x = 9007199254740992.0 = double(2^53)
# one = double(1.0)

fldl    x           # st(0) = 2^53
faddl   one         # st(0) = 2^53 + 1
fsubl   x           # st(0) = (2^53 + 1) - 2^53
= 1
fstpl   out         # out = 1.0
```

Demo: https://onlinegdb.com/oO0bvV2KJ

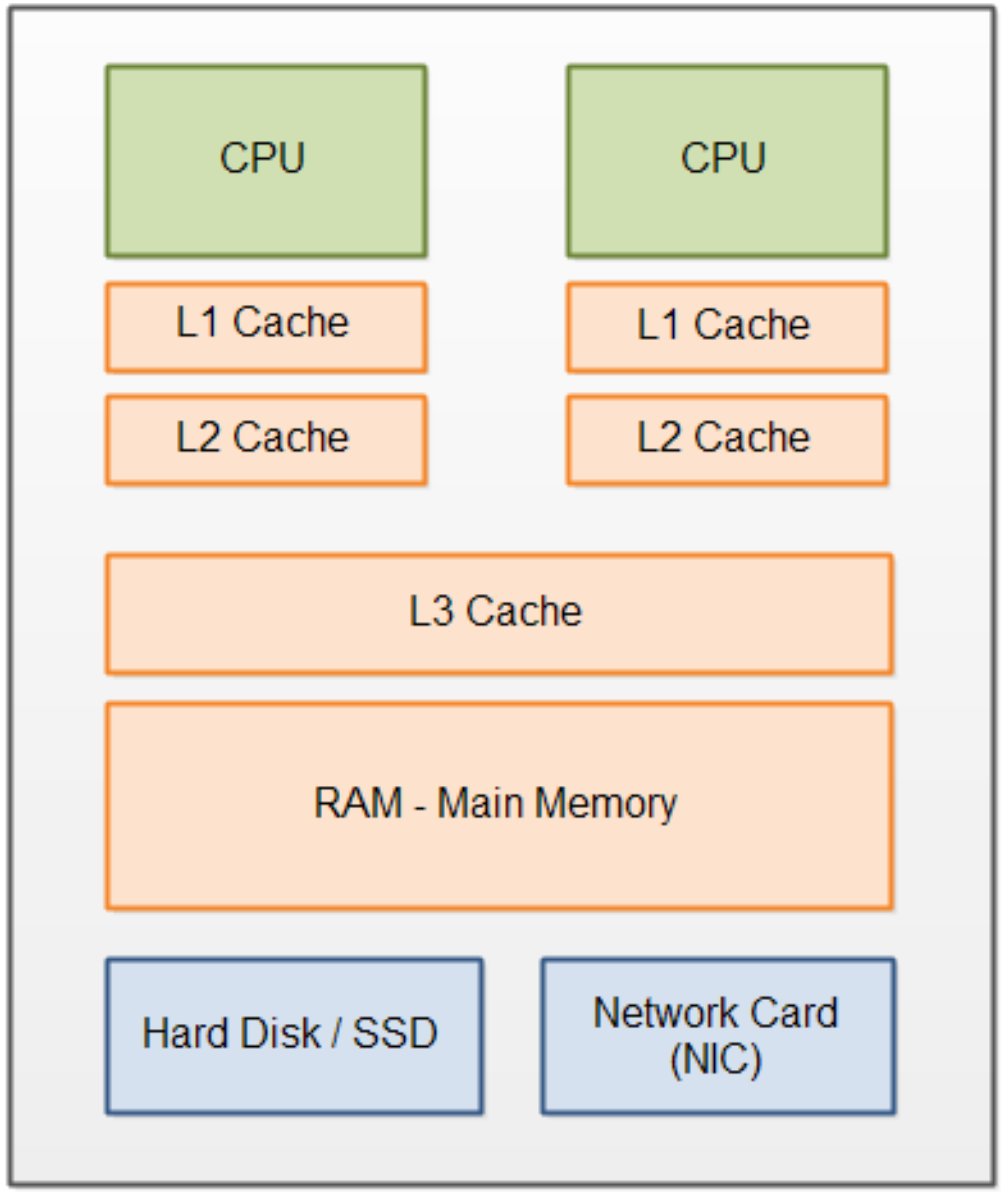
```
# x = 9007199254740992.0 =
double(2.53)
# one = double(1.0)

fldl    x           # st(0) = 2^53
faddl   one         # st(0) = 2^53 +
1
fstpl   tmp         # tmp = 2^53
fldl    tmp         # st(0) = 2^53
fsubl   x           # st(0) = 0
fstpl   out         # out = 0.0
```

PROCESSORS ARE TRYING TO TRICK YOU

Caching 101 – rough sketch

- Typically, a multicore system has some per-process cache and some shared cache.
- Caches can be represented as a 2D table where every entry stores a block of memory.

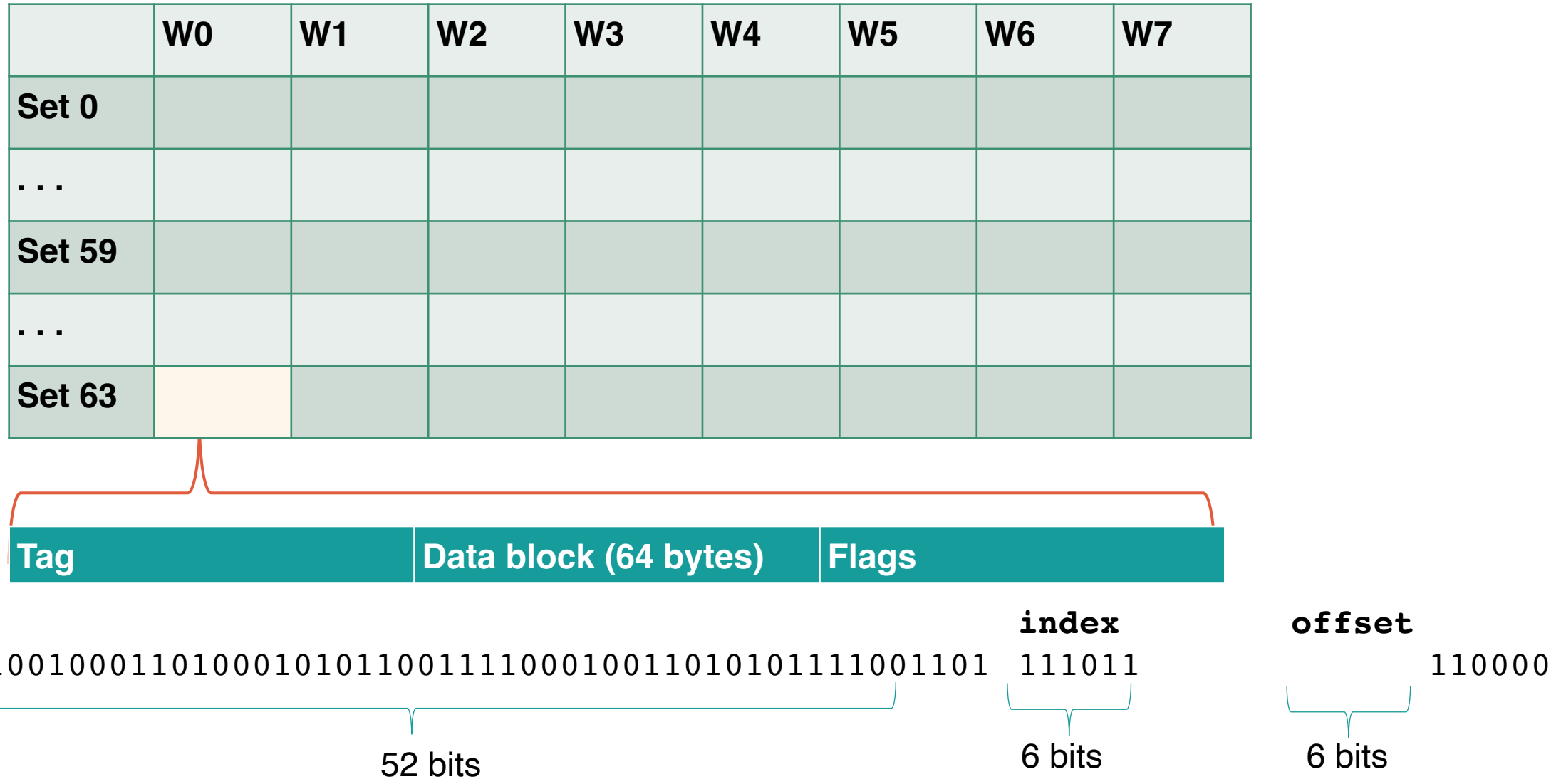


Caching 102 – 32KB cache example

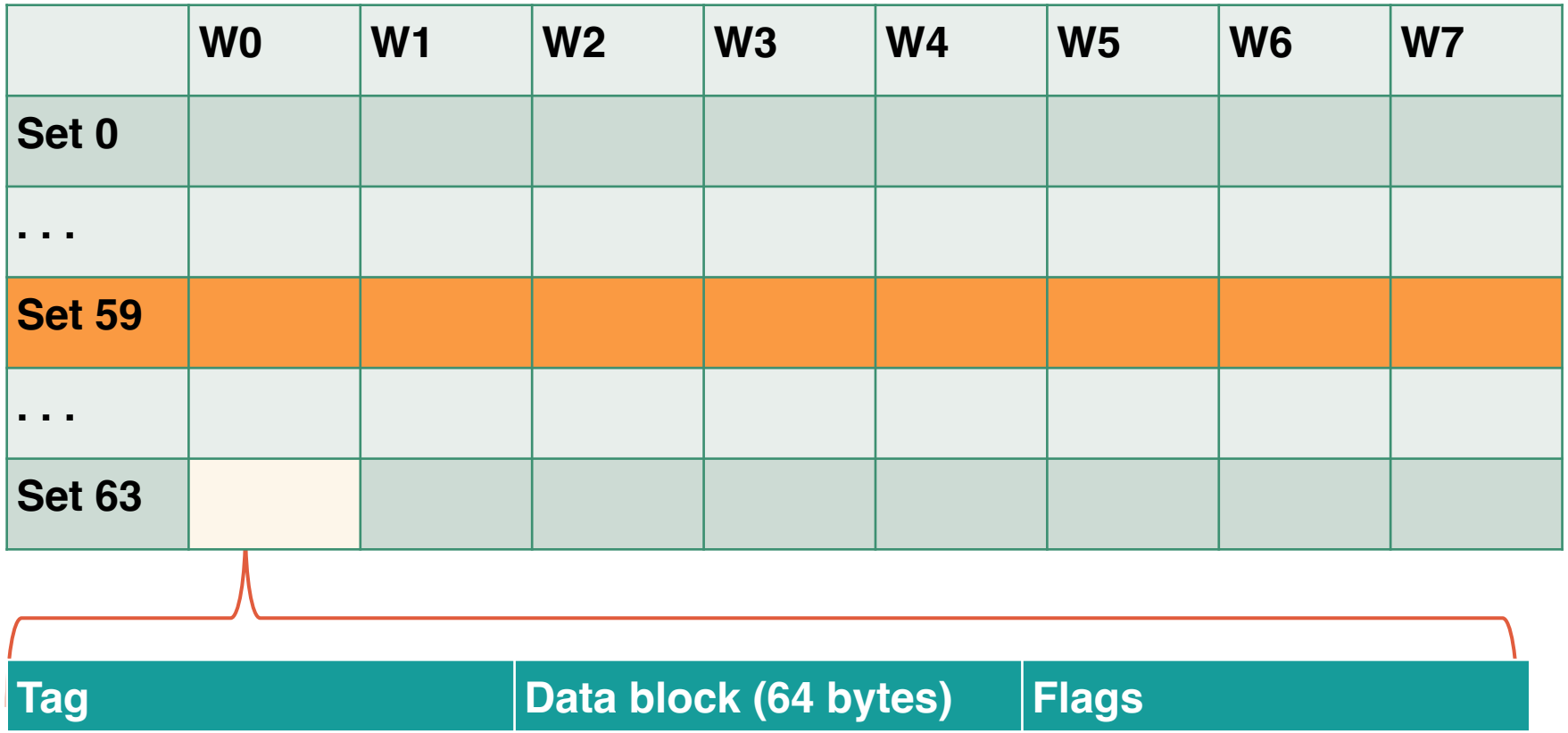
	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
Set 1								
Set 2	This is a line	and this is a line						
. . .								
Set 63			and this is a line					

- A cache **line** is selected using a transformation of requested memory address + lookup
- Every line represents a **64-byte chunk of memory**

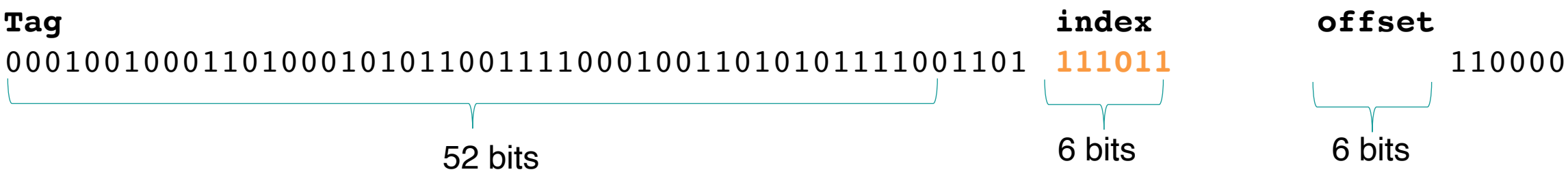
Caching 102 – 32KB cache example



Caching 102 – 32KB cache example



Step 1
Select a set using index bits.



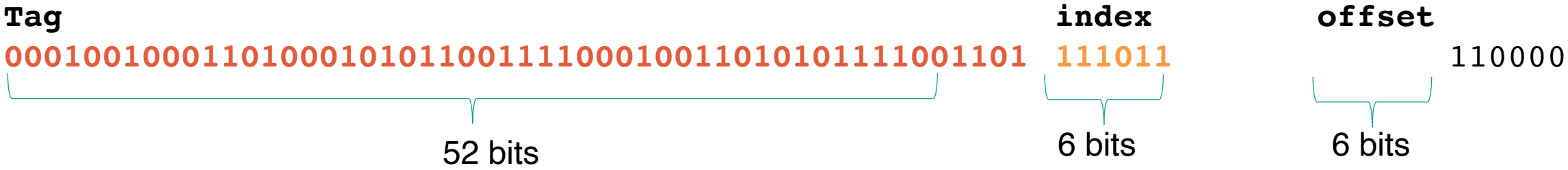
Caching 102 – 32KB cache example

	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59	tag=?	tag=?	tag=?	tag=?	tag=?	tag=?	tag=?	tag=?
. . .								
Set 63								

Step 2
Compare the tag with tags in all the ways.

If a tag matches, update that entry.

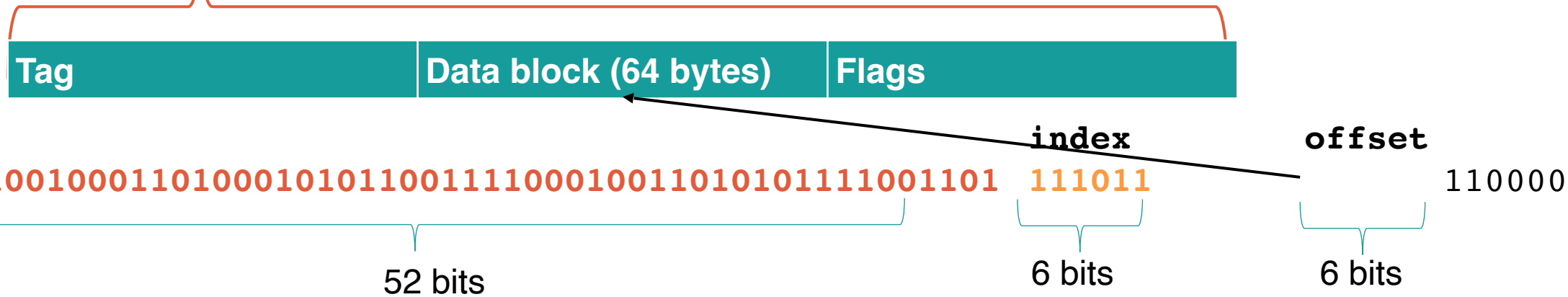
If no, evict.



Caching 102 – 32KB cache example

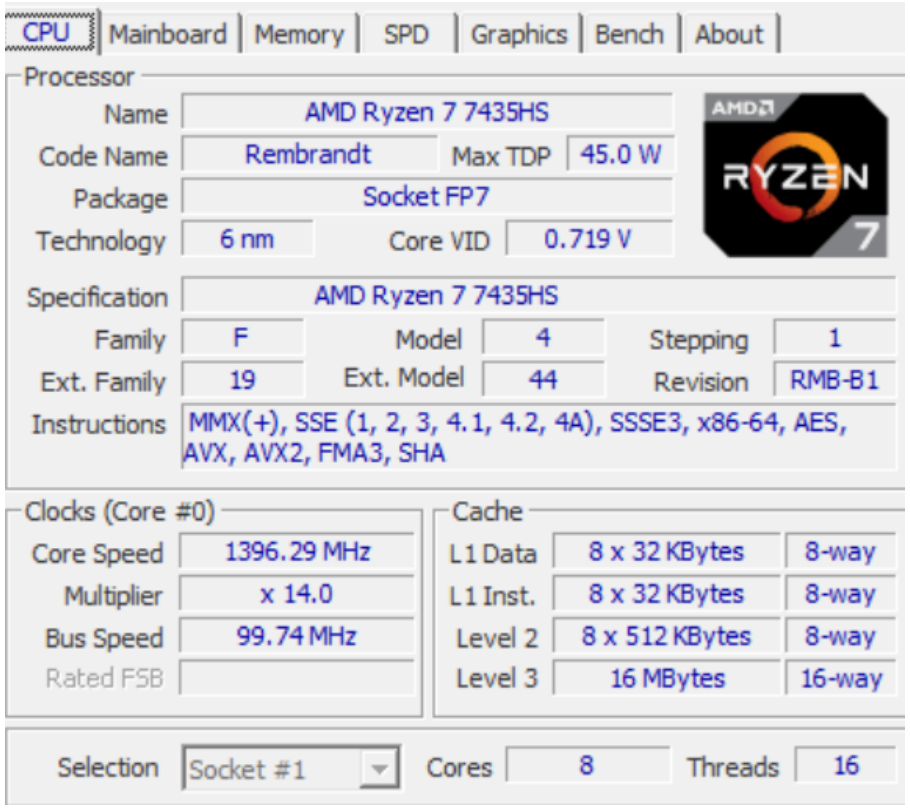
	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59	tag=?	tag=?	tag=?	tag=?	tag=?	tag=?	tag=?	tag=?
. . .								
Set 63								

Step 3
Inside the selected cache line, write memory address at offset.



Caching – summary

- With caching, some memory accesses are faster than other ones.
- For AMD 7435HS, L3 cache has ~10x less latency than RAM
- Sharing a cache opens up so many possibilities for attacks
- But you can attack even without a shared cache

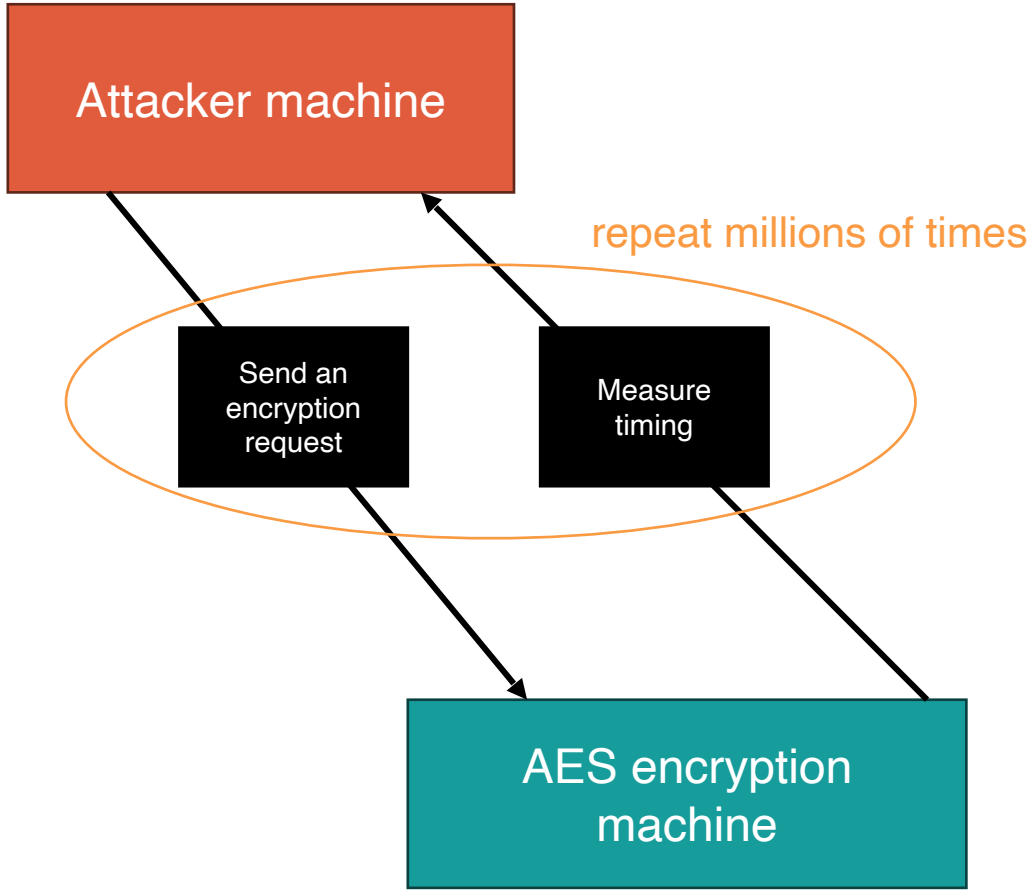


sponsored by https://www.cia.gov/

ATTACK TIME

Remote AES-256 cache timing attack

- AES-256 is a symmetric encryption algorithm
- There's a remote encryption server, you know its hardware specs, don't know the key.
- If you learn the key, the server is cooked
- OpenSSL's implementation of AES uses in-memory lookup tables
- Assume you can have the same CPU



https://cr.yp.to/antiforgery/cachetiming-20050414.pdf

Remote AES-256 cache timing attack

- AES-256 is a bit complicated, but the reference is on the right

AES-256 (simplified):

16-byte input *n*
16-byte key *k*
Constant 256-byte table $\mathrm{S} = (99, 124, 119, \ldots)$
Constant 256-byte table $S' = (198,248, 238,\ldots)$

Lookup tables (no need to remember):

$$T_0[b] = \left(S'[b], S[b], S[b], S[b] \oplus S'[b]\right)$$

$$T_1[b] = \left(S[b] \oplus S'[b], S'[b], S[b], S[b]\right)$$

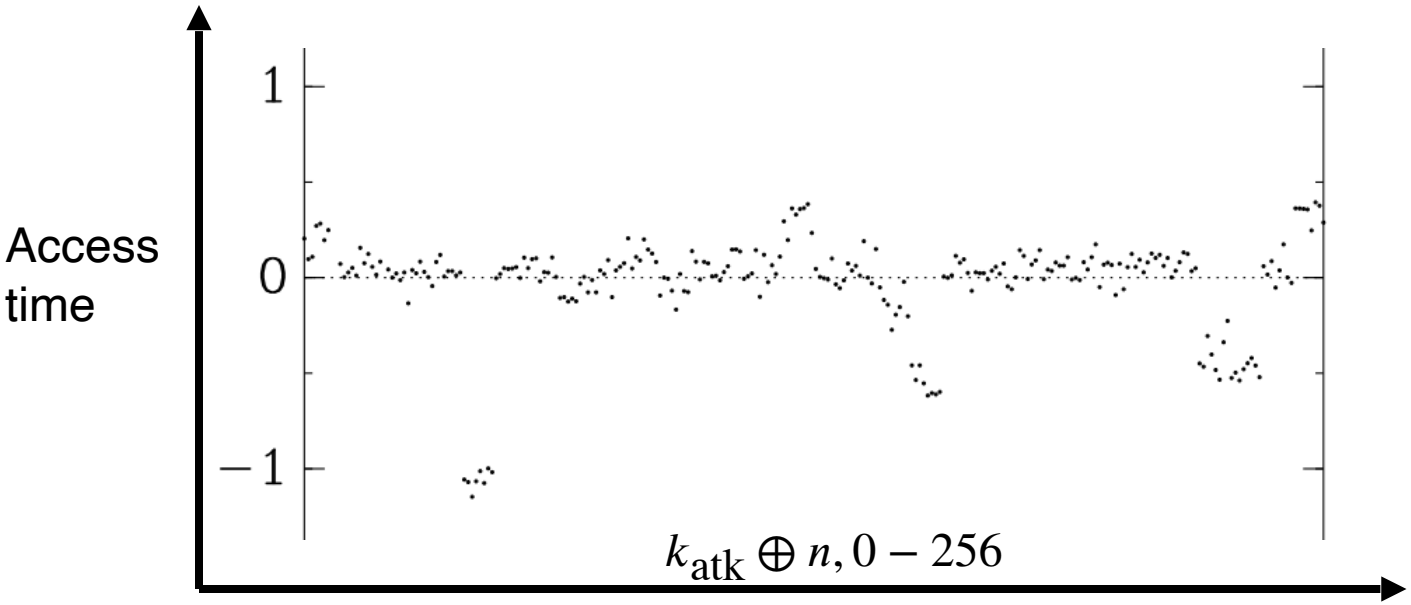
$$T_2[b] = \left(S[b], S[b] \oplus S'[b], S'[b], S[b]\right)$$

$$T_3[b] = \left(S[b], S[b], S[b] \oplus S'[b], S'[b]\right)$$

https://cr.yp.to/antiforgery/cachetiming-20050414.pdf

Remote AES-256 cache timing attack

- Run AES-256 on the same hardware with key $k_{\text{atk}} = 0$
- Run that for many possible values of *n*, (many times) and measure how long that takes on average. Get a distribution like



AES-256 (simplified):

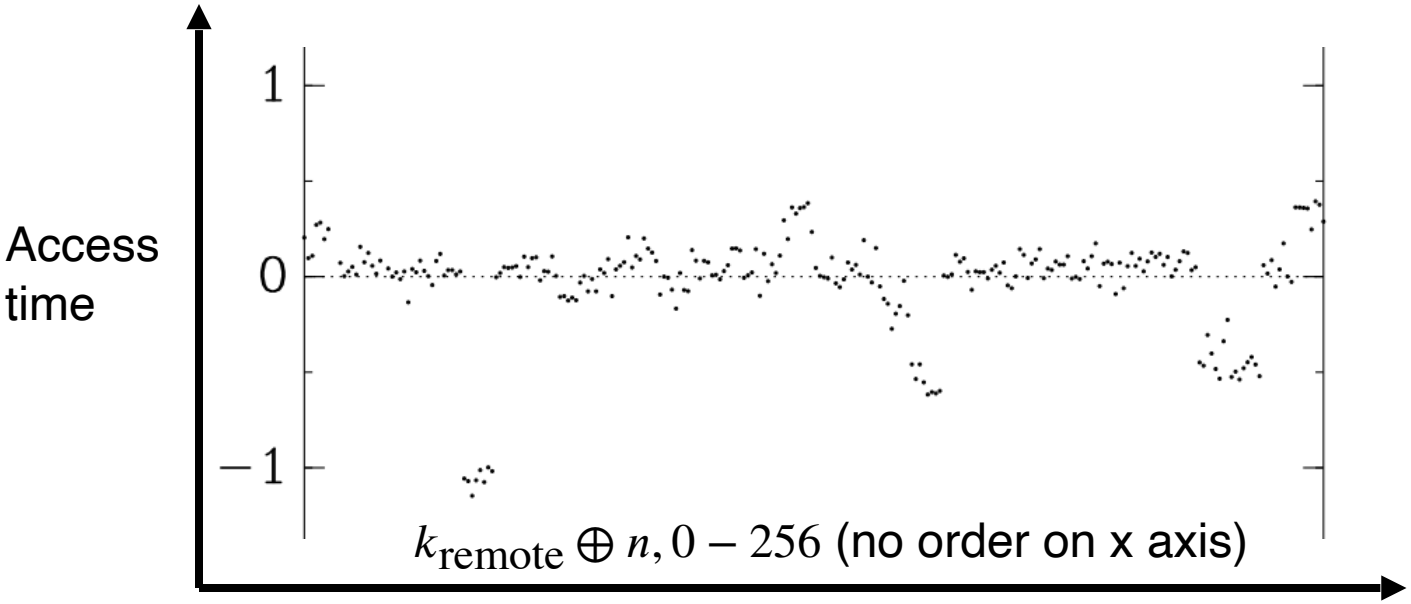
1-byte input *n*
1-byte key *k*

Lookup table $\mathrm{T[n \oplus k]}$

https://cr.yp.to/antiforgery/cachetiming-20050414.pdf

Remote AES-256 cache timing attack

- Now, send caching requests to the remote machine and get a similar distribution (but shifted)! Then, just infer k_{remote} through XOR
- Remote server is cooked



AES-256 (simplified):

1-byte input *n*
1-byte key *k*

Lookup table $\mathrm{T[n \oplus k]}$

https://cr.yp.to/antiforgery/cachetiming-20050414.pdf

Now, the real question

We know that cache can be shared between cores. How do we exploit that?

Basic attack: Flush+Reload

- Say, two processes memory. Attacker can only read the memory
- Attacker wants to know which memory address victim recently accessed

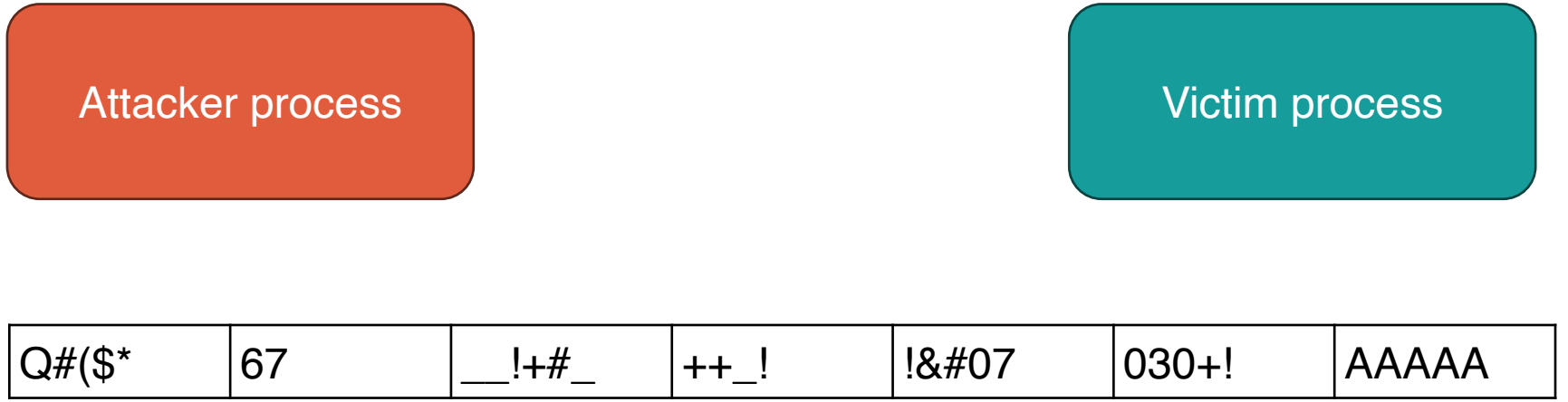
```
#include <stdint.h>

static const uint8_t table[256] = {
    /* 256 bytes */
};

uint8_t lookup_secret(uint8_t secret, uint8_t input) {
    return table[secret ^ input];
}
```

Basic attack: Flush+Reload

- Say, two processes memory. Attacker can only read the memory
- Attacker wants to know which memory address victim recently accessed



Basic attack: Flush+Reload

- Say, two processes memory. Attacker can only read the memory
- Attacker wants to know which memory address victim recently accessed

Step 1: attacker flushes the processor cache

Basic attack: Flush+Reload

- Say, two processes memory. Attacker can only read the memory
- Attacker wants to know which memory address victim recently accessed

Step 2: victim process accesses some memory address -> it gets cached

Basic attack: Flush+Reload

- Say, two processes memory. Attacker can only read the memory
- Attacker wants to know which memory address victim recently accessed

Step 3: attacker reads through all the memory addresses and measures access times

Basic attack: Flush+Reload

- Say, two processes memory. Attacker can only read the memory
- Attacker wants to know which memory address victim recently accessed

Step 4: attacker flushes the cache again and repeats the process

Flush+Reload is still not fixed

- Very problematic for shared crypto libraries
- Can’t really be fixed by hardware methods because shared caches are essential
  - Okay, there’s Intel CAT (partitions ways between processes)
  - And there’s stuff like hardware AES instructions
  - But those are not everywhere and not every library uses them
- So, most mitigations are actually software ones

Demo!

Go here: https://onlinegdb.com/BAkaGHLaM

What if you don’t share memory between processes?

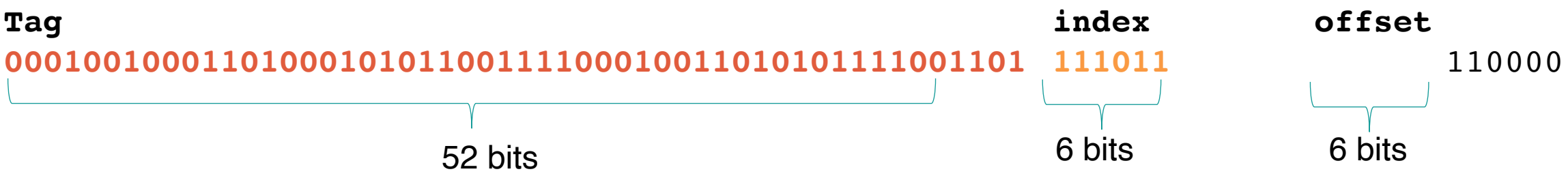
There’s still a way

Prime+Probe

Remember this guy? This is him now. Feel old yet?

- Suppose we want to know if some victim process accesses memory address **&a**

	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59		&a						
. . .								
Set 63								



Prime+Probe

We know that the address &a is supposed to be in set 59…

Prime: Pick a bunch of memory addresses that'd be in set 59 and have different tags, flood the cache line with our own attacker addresses.

	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59	A0	A1	A2	A3	A4	A5	A6	A7
. . .								
Set 63								

tag, 52 bits

index, 6 bits.
must be same as in &a

offset(doesn't matter)

A0= [tag0] **111011**
110000
A1= [tag1] **111011**
110000

Prime+Probe

We know that the address &a is supposed to be in set 59…

Prime: Pick a bunch of memory addresses that'd be in set 59 and have different tags, flood the cache line with our own attacker addresses.

	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59	A0	A1	A2	A3	A4	A5	A6	A7
. . .								
Set 63								

tag, 52 bits

index, 6 bits.
must be same as in &a

offset(doesn't matter)

A0= [tag0] **111011**
110000
A1= [tag1] **111011**
110000

Prime+Probe

We know that the address &a is supposed to be in set 59…

Victim access: Victim accesses address **&a** address and **A1** gets evicted

	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59	A0	&a	A2	A3	A4	A5	A6	A7
. . .								
Set 63								

tag, 52 bits

index, 6 bits.
must be same as in &a

offset(doesn't matter)

A0= [tag0] **111011**
110000
A1= [tag1] **111011**
110000

Prime+Probe

We know that the address &a is supposed to be in set 59…

Probe: we scan our addresses, measuring access times! Kinda similar to flush+reload.

	W0	W1	W2	W3	W4	W5	W6	W7
Set 0								
. . .								
Set 59	A0	&a	A2	A3	A4	A5	A6	A7
. . .								
Set 63								

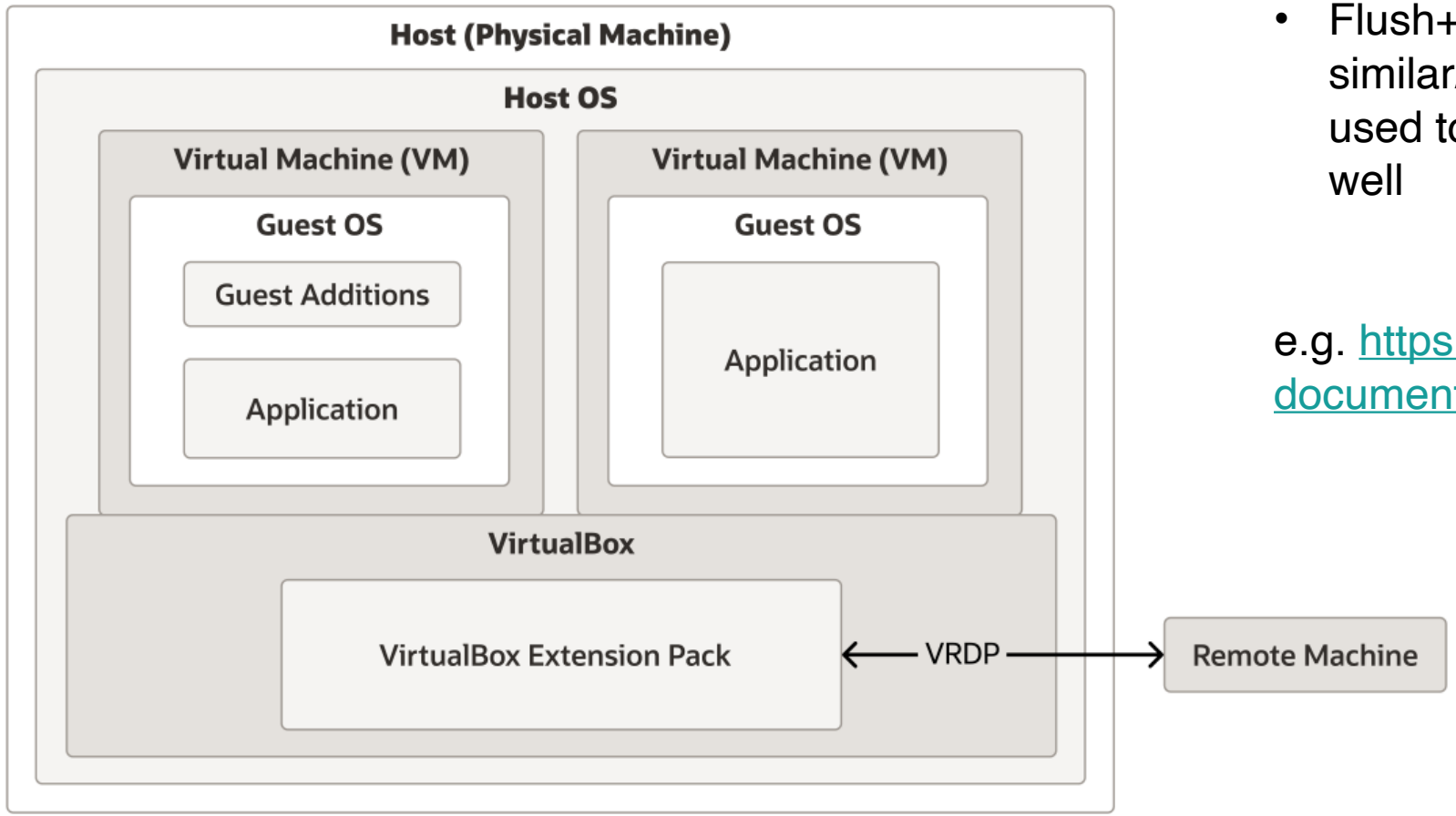
tag, 52 bits

index, 6 bits.
must be same as in &a

offset(doesn't matter)

A0= [tag0] 111011
110000
A1= [tag1] 111011
110000

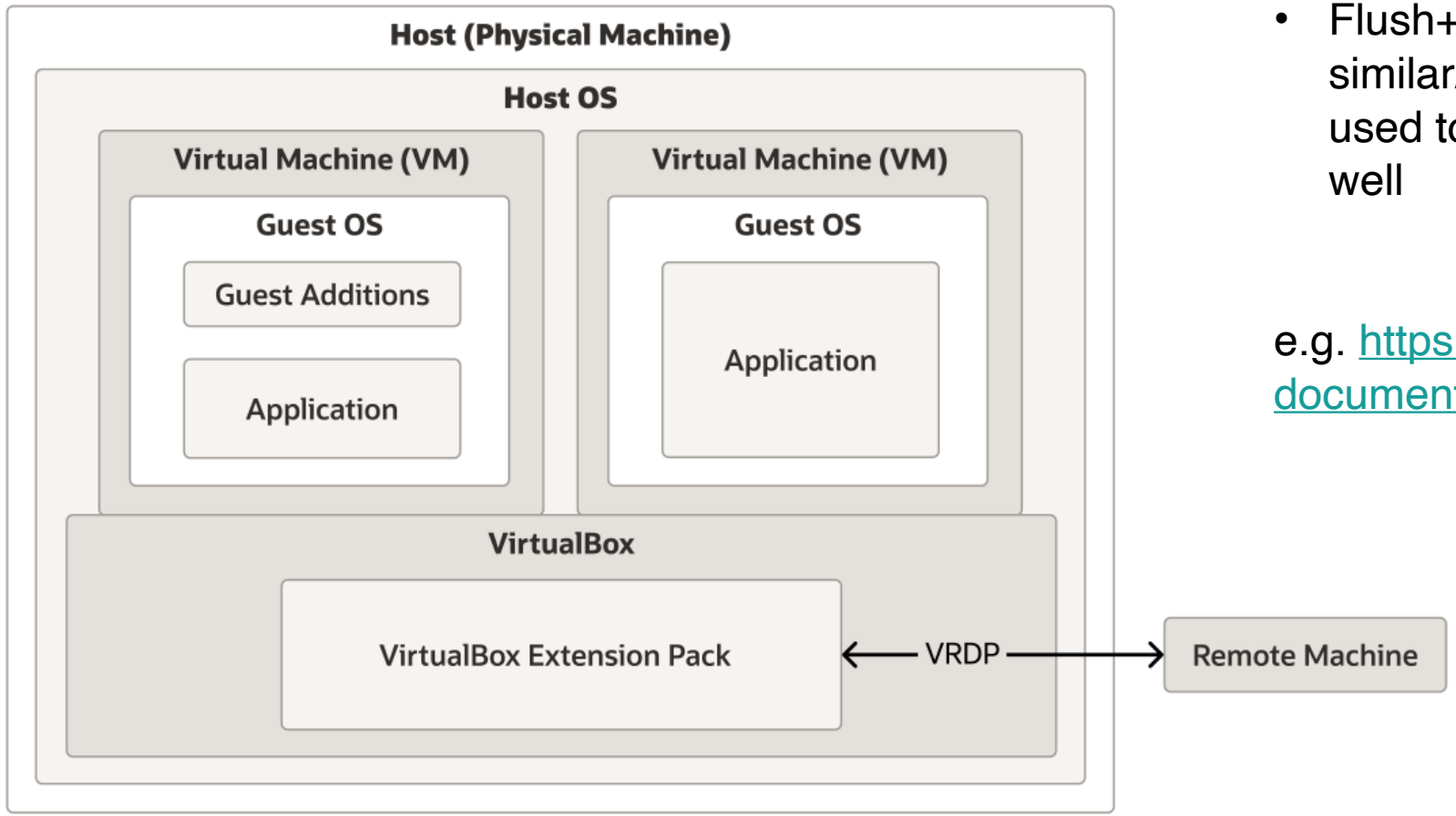
Cross-VM cache attacks



- Flush+Reload and Prime+Probe (and similar/derived vulnerabilities) can be used to exploit virtual machines as well

e.g. https://ieeexplore.ieee.org/document/7447807

Cross-VM cache attacks



- Flush+Reload and Prime+Probe (and similar/derived vulnerabilities) can be used to exploit virtual machines as well

e.g. https://ieeexplore.ieee.org/document/7447807

“caches are dope but a man must not live by caches alone”

- Confucius during the siege of London, 1471

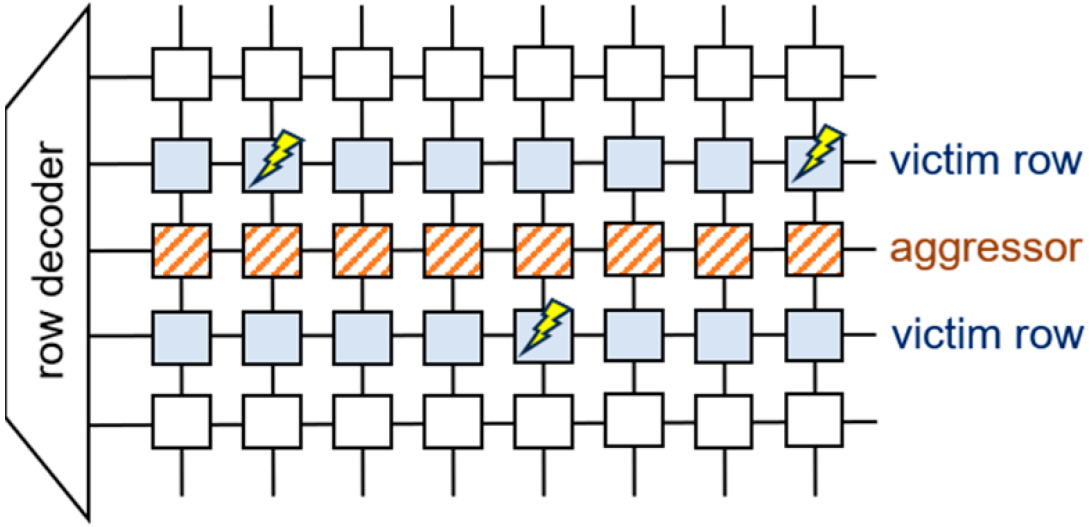
Side quest: DRAM Rowhammer attack

Idea: if you hammer DDR3 SDRAM hard enough, you might be able to flip some bits you're not supposed to have access to.

Why this works? IDK. This vulnerability is not really about processors ¯_(ツ)_/¯

You can read more here: https://dl.acm.org/doi/abs/10.1145/2678373.2665726

(or just here https://en.wikipedia.org/wiki/Row_hammer)




```
hammer:
    mov (X), %eax     // read from address X
    mov (Y), %ebx     // read from address Y
    clflush (X)       // flush cache for address X
    clflush (Y)       // flush cache for address Y
    mfence
    jmp hammer
```

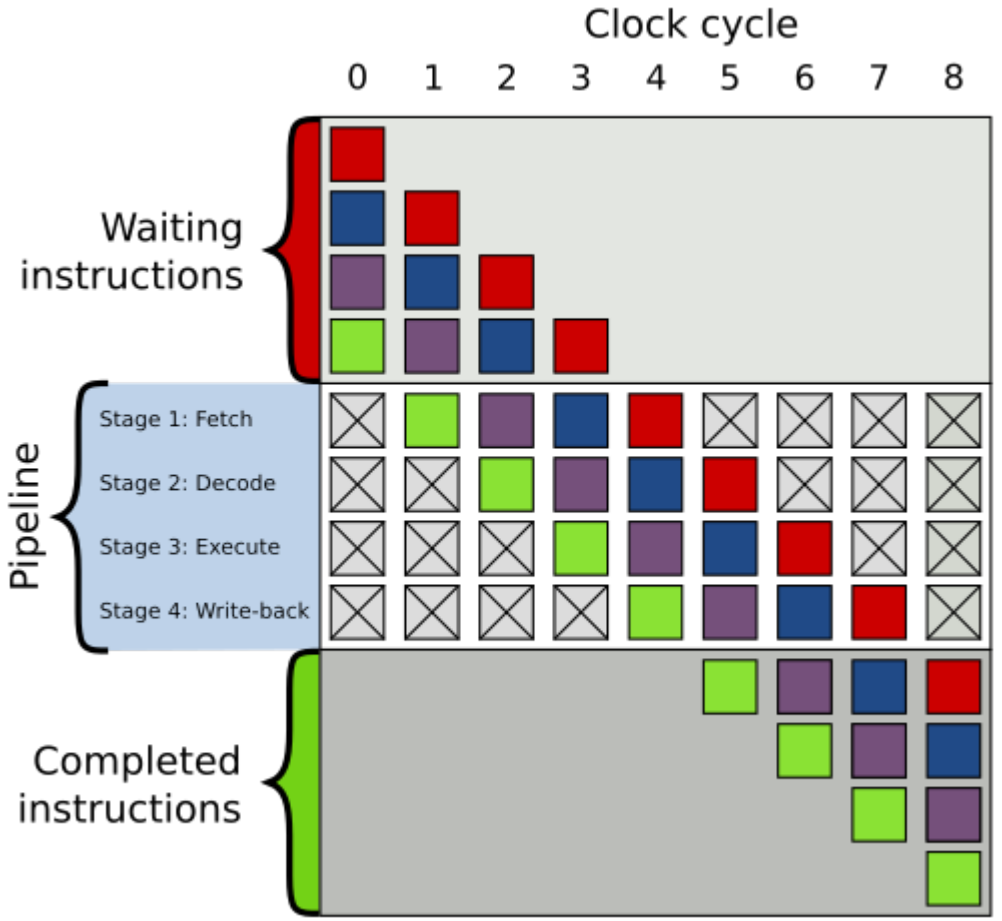
1-800-CALL-FBI (225-5324)

Microarchitectural attacks

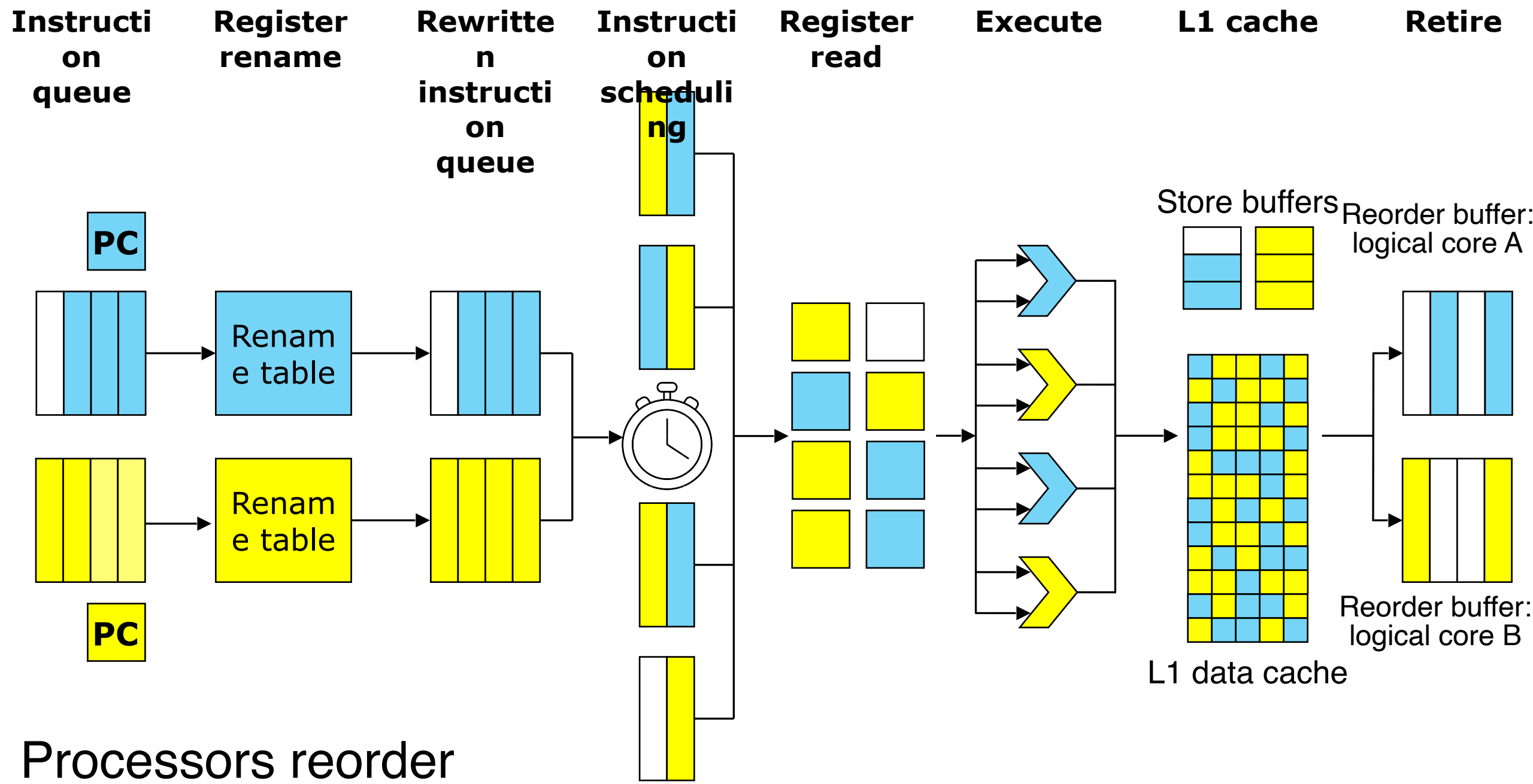
Wtf is microarchitecture

(as if architecture is not hard enough)

- *Basically, all the fancy ways in which Intel/AMD/ Apple etc. try to speed up performance*
- First of all, processors pipeline instructions. Roughly:
  - Fetch the instruction
  - Decode the instruction
  - Execute the instruction
  - Write back the results
- This increases clock speed and forces you to build processors in more modular way



Visualization of pipelining: https://timmastny.com/blog/visualizing-cpu-pipelining/



Processors reorder instructions!

*courtesy of CS 2630 (everybody MUST take it)

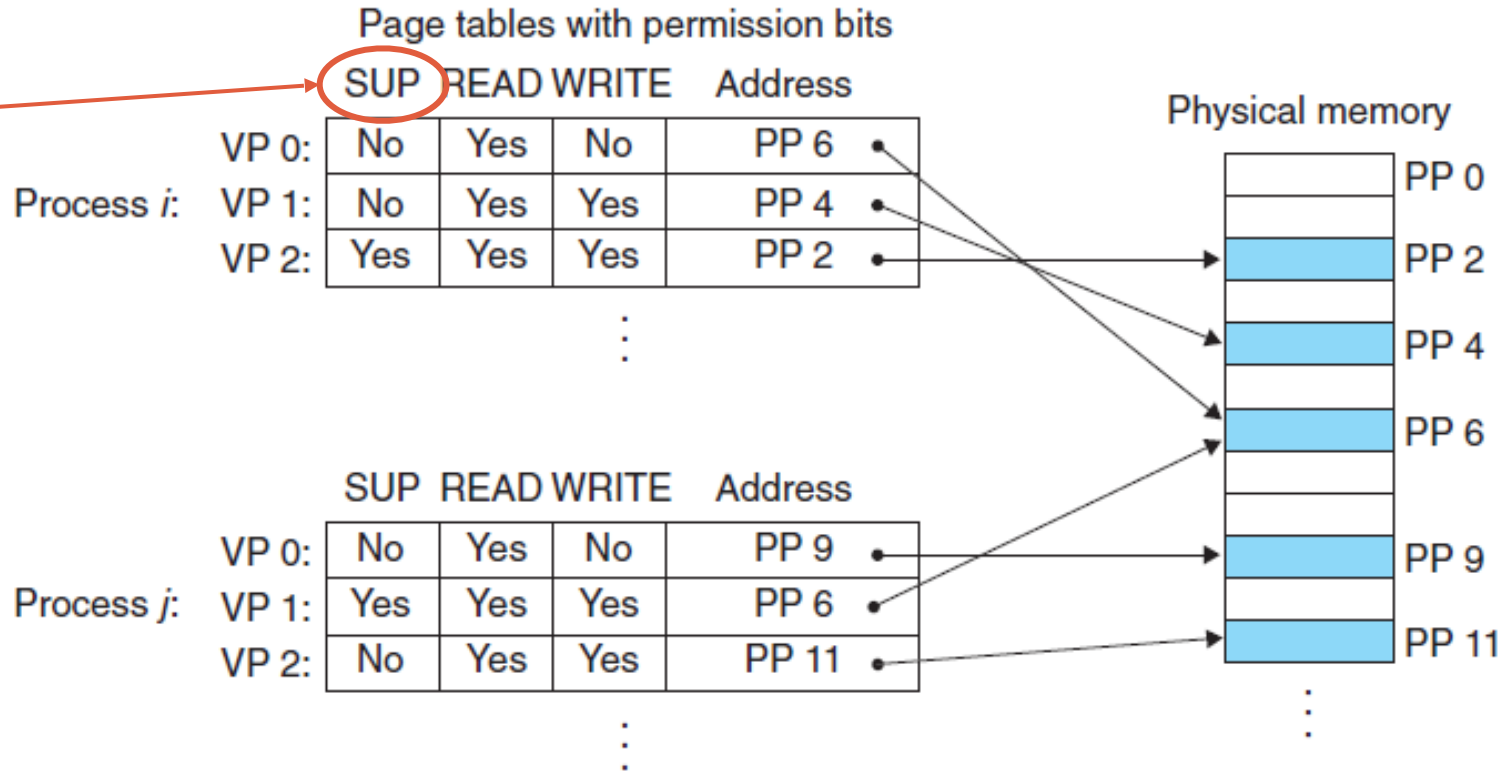
Boom, MELTDOWN

- This is a vulnerability that allows you to read kernel space. JUST LIKE THAT
- First, remember about virtual memory page permissions

Privilege bit

When you try to access a privileged address as a user space process, you get an exception!

**at least that's what's supposed to happen*



Boom, MELTDOWN

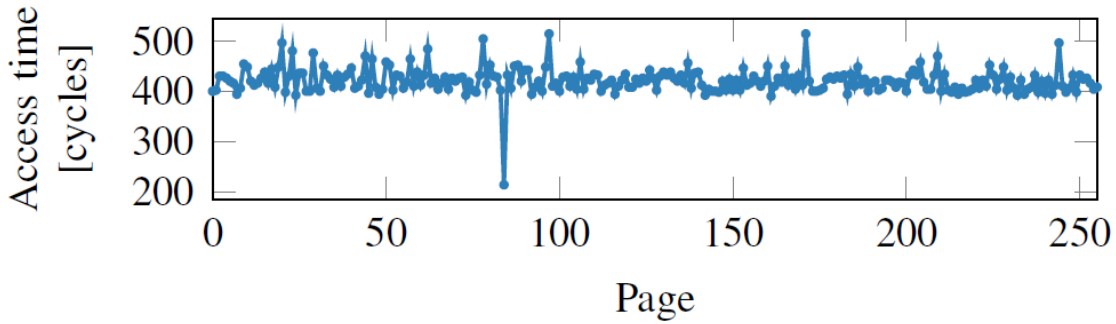
*again, courtesy of CS 2630

```
(1) char probe_array[256 * BYTES_IN_MEM_PAGE];
(2) clflush(probe_array); //Flush all cache lines in array
(3) char secret = *kernel_addr;
(4) char ignored = probe_array[secret * BYTES_IN_MEM_PAGE];
```

Normally, you'd get an exception at line (3) because you trying to access a kernel address you're not allowed to access.

This will happen indeed.

However, the processor loads `probe_array[secret * BYTES_IN_MEM_PAGE]` **BEFORE** the permission check happens.



Now, if you iterate through `probe_array`, and time every read, you have the secret revealed!

How to get rid of MELTDOWN?

- Standard fix: kernel address randomization
- Kernel pagetable isolation (Linux)
  - For user space pagetables, keeps only the minimal number of kernel mappings
  - Makes TLB lookups slower!
  - But not that much

Anyways, it’s a thing of the past…

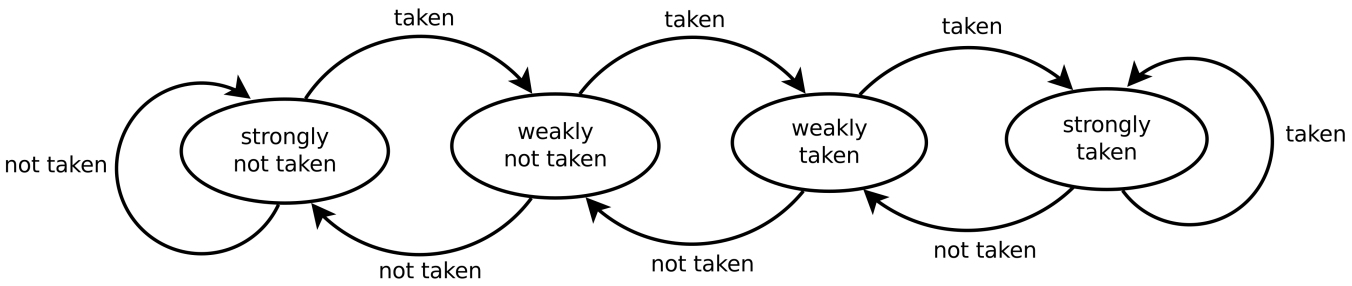
That’s interesting.

What about branch prediction?

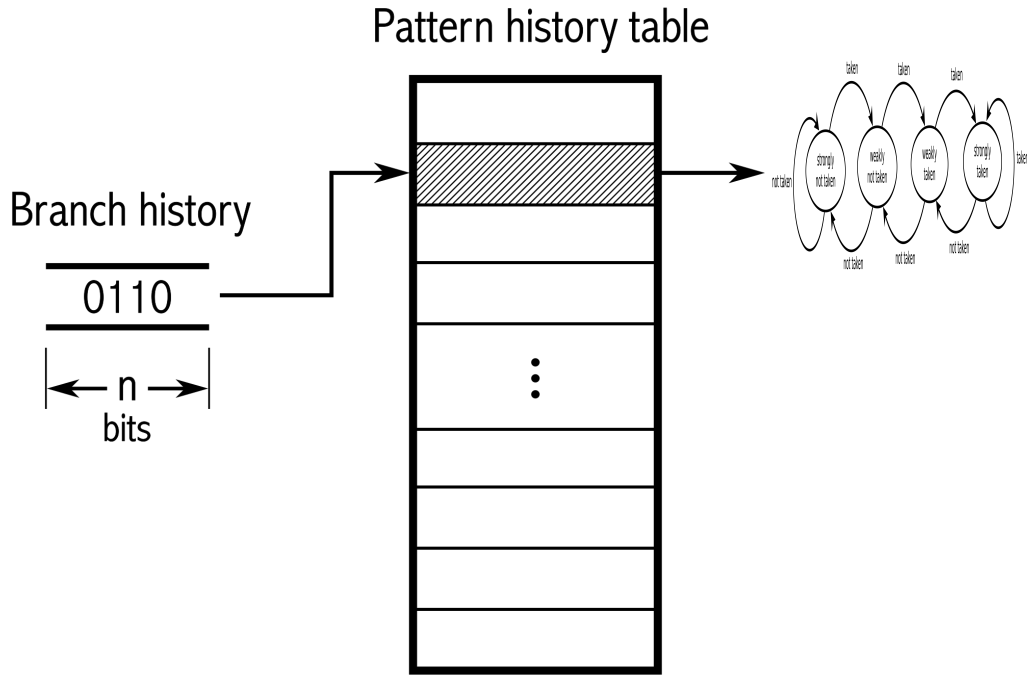
Branch predictors

- They are pieces of code that try to predict next jump
- They can be more fancy or less fancy
- For every branch, the predictor tries to guess outcome and prefetch necessary data

One-level branch predictor



Two-level branch predictors

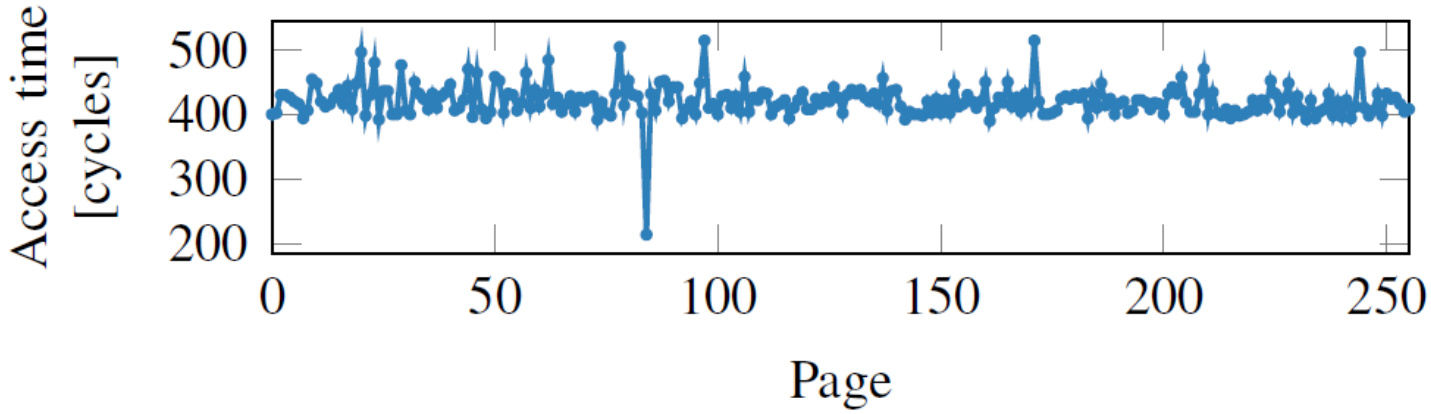


Branch predictors – SPECTRE

```
(1) char probe_array[256 * BYTES_IN_MEM_PAGE];
(2) clflush(probe_array); // flush all cache lines in array
(3) mistrain branch predictor on: if (x < array_len)
(4) if (malicious_x < array_len) {     // predicted true, actually false
(5)     char secret = array[malicious_x];
(6)     char ignored = probe_array[secret * BYTES_IN_MEM_PAGE];
(7) }
```

Same th

Never re



ed to be accessed.

How do we fix it?

- We don’t lol

```
heorhii@h-legion:~$ cat /proc/cpuinfo | grep spectre
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
bugs            : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass srso
```

…and that's where it's going

Transient execution CPU vulnerability

Article Talk

From Wikipedia, the free encyclopedia

(Redirected from Speculative execution CPU vulnerabilities)

> *Main articles: Downfall (security vulnerability), Foreshadow, Lazy F vulnerability), Microarchitectural Data Sampling, Retbleed, Spectre SWAPGS (security vulnerability)*

Transient execution CPU vulnerabilities are vulnerabilities in which are executed temporarily by a microprocessor, without committing thei secret data to an unauthorized party. The archetype is Spectre, and tra attack category, one of several categories of side-channel attacks. Sin have been identified.

Overview [edit]

Modern computers are highly parallel devices, composed of componer operation (such as a branch) cannot yet be performed because some completed, a microprocessor may attempt to *predict* the result of the e acting as if the prediction were correct. The prediction may be based c operation completes, the microprocessor determines whether the pred proceeds uninterrupted; if it was incorrect then the microprocessor roll

Contents hide